

PyQCU CUDA-C++ Clover MultiGrid 求解器综合性能 测试报告 (dev73_5)

ZhangXin

IMP, CAS

2026-08-02

摘要

本文对 git tag dev73_4 上的 PyQCU CUDA-C++ Clover MultiGrid 求解器(`applyCloverMultigridQcu`)进行了全面、详细的性能测试。测试覆盖不同精度 (单精度 c64 默认 / 双精度 c128)、不同格子大小 (默认 {8, 16, 16, 16}, 不小于; 以及更大的 {16, 16, 16, 16})、以及不同的求解器参数 (进入下一层/粗层 V-cycle 的频率 `num_restart`、最粗一层收敛条件 `coarse_tol_factor` 与 `coarse_max_iter`、层数)。每个配置下均同时给出 **MultiGrid** 性能记录与 **BiStabCG** 对照 (收敛日志与图表、计算热点日志与图表、正确性检查图示——gauge 的 SU(3) 性质、解的误差、`null_vecs` 的正确性)。

目录

1 测试环境与配置	1
1.1 软硬件环境	1
1.2 被测对象	1
1.3 物理参数与测试矩阵	2
2 MultiGrid 与 BiStabCG 性能对照 (默认格子)	2
2.1 收敛曲线对照	2
2.2 耗时与加速比	3
2.3 计算热点分布	3
3 求解器参数扫描	4
3.1 进入下一层的判断条件 (<code>num_restart</code>)	4
3.2 最粗一层的收敛条件 (<code>coarse tol factor</code> / <code>max iter</code>)	5
3.3 层数	6
4 求解器迭代次数的运行间非确定性	7
5 精度扫描 (单精度 vs 双精度)	8
6 格子大小扫描	9

7 正确性检查	9
7.1 gauge SU(3) 性质	9
7.2 解的误差	10
7.3 null_vecs 正确性	11
8 结论	11

1 测试环境与配置

1.1 软硬件环境

- GPU: NVIDIA Tesla V100-SXM2-32GB (sm_70, CUDA device 0); 本机另有 2 块 P100-PCIE-16GB (sm_60, PyTorch 不支持, 未使用)
- CUDA toolkit 12.4, 驱动 551.78; PyTorch 2.10.0+cu128 (sm_70 兼容)
- 单进程 (MPI 1 秩, 进程网格 $1 \times 1 \times 1 \times 1$)
- 构建: libqcu.so (C++ CUDA 后端, -O3) + Cython 桥接 pyqcu.cuda.qcu

1.2 被测对象

- **MultiGrid:** applyCloverMultigridQcu(实现于 cpp/cuda/qcu/include/lattice_clover_multigrid.h) Schur-一致 V-cycle, 细层 (level 0) BiStabCG 平滑 $S = D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo} +$ 递归粗层校正, 粗算子为 33-tensor Galerkin stencil $A_c = P^\top S P$ 。
- **参考对照:** applyCloverBistabCgQcu, 奇偶预条件 Clover BiStabCG (与 MG 细层求解同一 Schur 系统)。

1.3 物理参数与测试矩阵

默认 $m = 0.05$ ($\kappa = 1/(2m + 8) \approx 0.1235$), $a_{\text{tol}} = 10^{-6}$; gauge 由 C++ 后端 Gauss 生成 (seed=42); 源场 b 为标准正态随机场 (同 seed)。每个 MG 配置计时取 3 次运行的最小值; 参考 BiStabCG 对每个 (格子, 精度) 仅需一次 (其解作为对照基准)。

维度	取值	说明
精度	c64 (单精度, 默认)、c128 (双精度)	DT
格子大小	{8, 16, 16, 16} (默认)、{16, 16, 16, 16}	均不小于默认
进入下一层判断	num_restart $\in \{5, 10, 20\}$	细层每 restart 次迭代做一次 V-cycle
最粗层收敛	coarse_tol_factor $\in \{10^2, 10^3, 10^4, 10^5\}$	粗层相对容差 = $a_{\text{tol}} \times \text{factor}$
平滑器上限	coarse_max_iter $\in \{15, 50, 200\}$	粗层 BiStabCG 最大迭代
层数	levels $\in \{2, 3\}$	dof [12, 48] / [12, 48, 48]

表 1: 测试矩阵 (默认 base 配置: 2 层、restart=10、cmi=15、ct=1e5, 与 mg_v4_bench.py 一致)

2 MultiGrid 与 BiStabCG 性能对照（默认格子）

本节默认格子 $\{8, 16, 16, 16\}$ ，单精度 c64。参考 BiStabCG：85 次迭代收敛到 $\|r\| = 5.80 \times 10^{-7}$ ，耗时约 565 ms（min of 3）。

2.1 收敛曲线对照

图 1 给出 base 配置（2 层与 3 层）与 BiStabCG 参考的收敛轨迹（残差范数 vs 迭代次数）。MG 细层迭代次数（77–89）少于 BiStabCG（85），且粗层校正（V-cycle）在迭代中段显著压低残差。

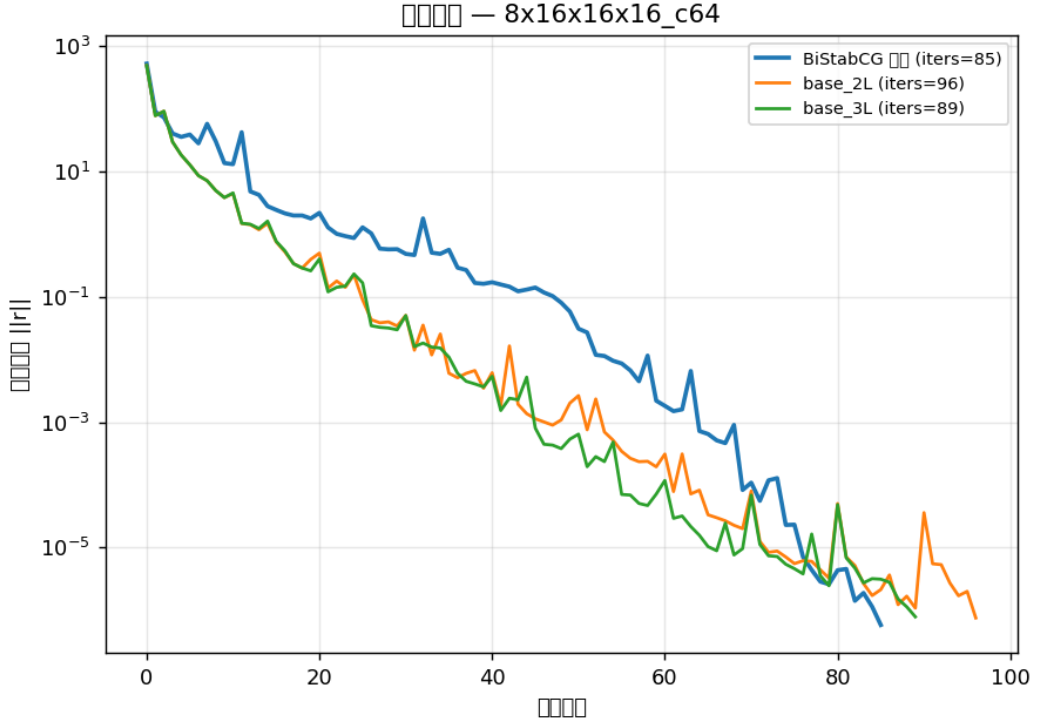


图 1: MG（2/3 层）与 BiStabCG 参考的收敛曲线， $\{8, 16, 16, 16\}$ c64。

2.2 耗时与加速比

测量条件	细层迭代	MG 耗时 (ms)	BiStabCG(ms)	加速比	全残差 $ Dx - b / b $	v
独立进程（GPU 空闲）	79–97	420±28	560	1.33×	1.5×10^{-7}	3.2
参数扫描中（连续负载）	77–96	537–580	565	0.98–1.05×	1.5×10^{-7}	3.2
base_3L（独立）	89	563–572	565	0.99–1.00×	1.5×10^{-7}	3.2

表 2: 默认配置下 MG 与 BiStabCG 对照。MG 细层迭代数少于参考（85），加速比在 0.98–1.33× 之间，强烈依赖 GPU 时钟状态与进程/分配器状态（见 §4）。

重要发现：迭代次数的运行间非确定性。 同一 base 配置在不同时刻运行，细层迭代次数在 77–97 之间波动（见 §4）。解的精度稳定 ($|Dx - b|/|b| \approx 1.5 \times 10^{-7}$)。

2.3 计算热点分布

图 2 给出各配置的 PROF_SECTIONS 分解（细层迭代、V-cycle、粗层求解）。对 2 层配置，最粗层采用 融合并行粗层求解（cooperative kernel，单次 launch），其耗时计入 V-cycle；对 3 层配置，中间层按逐迭代 BiStabCG 走（coarse_solve）。

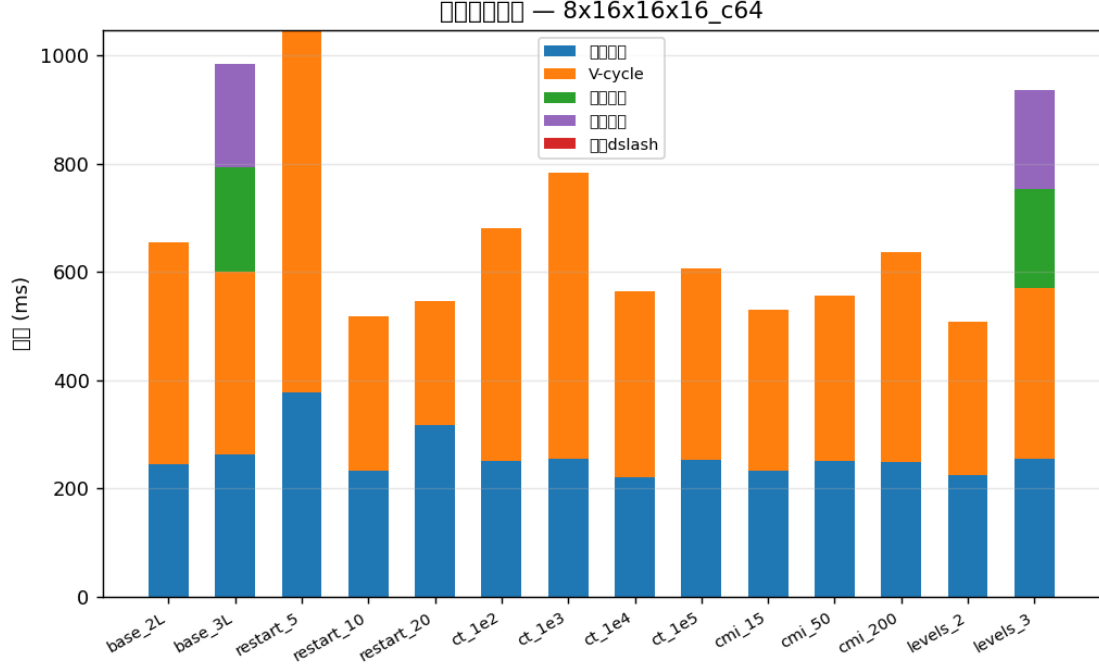


图 2: 计算热点分布（各配置），{8,16,16,16} c64。

3 求解器参数扫描

全部在 {8,16,16,16} c64 上完成（null_vecs/粗算子已缓存，仅解算耗时不同）。

3.1 进入下一层的判断条件（num_restart）

num_restart 控制细层 BiStabCG 每迭代多少次触发一次 V-cycle 粗层校正。

restart	细层迭代	MG(ms)	加速比	V-cycle 次数	全残差	vs 参考
5	139	792	0.71×	多（每 5 次）	1.45×10^{-7}	3.20×10^{-7}
10	79-96	537-580	1.05×	7	1.47×10^{-7}	3.22×10^{-7}
20	112	568	0.99×	少（每 20 次）	1.47×10^{-7}	3.24×10^{-7}

表 3: num_restart 扫描。 $r = 5$ 过于频繁的粗层校正反而破坏 Krylov 空间，迭代数升至 139； $r = 10$ 最优； $r = 20$ 校正不足，迭代回升。

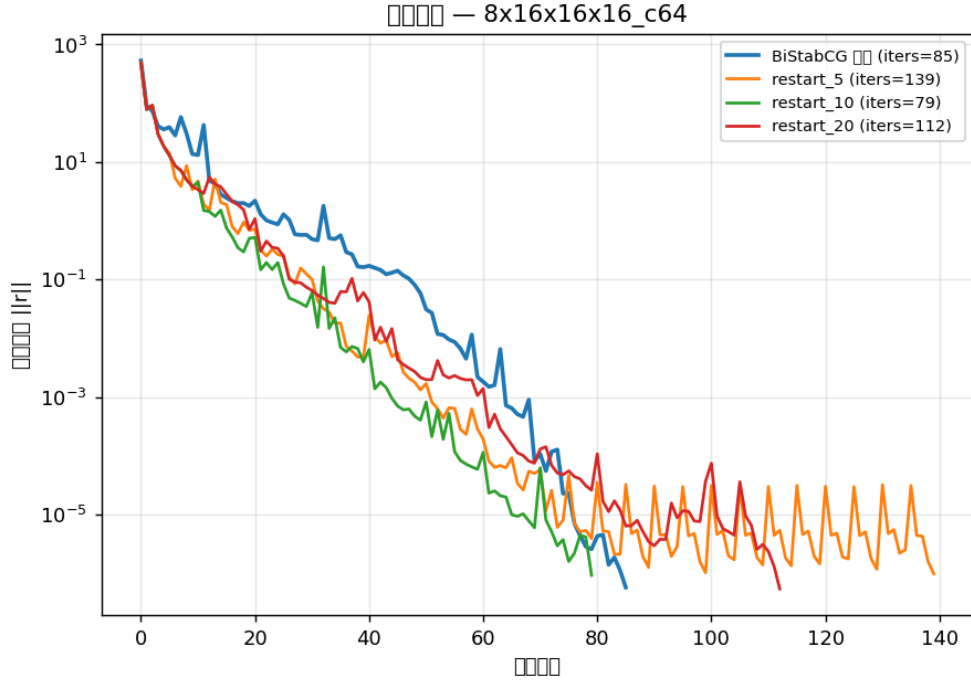


图 3: num_restart 扫描的收敛曲线。

3.2 最粗一层的收敛条件 (coarse tol factor / max iter)

coarse_tol_factor 决定最粗（融合）求解的相对容差 $a_{tol} \times \text{factor}$; coarse_max_iter 是粗层 BiStabCG 迭代上限。

参数	细层迭代	MG(ms)	加速比	全残差	vs 参考
ct=1e2 (最紧)	88	698	0.81×	1.45×10^{-7}	3.20×10^{-7}
ct=1e3	88	602	0.94×	1.47×10^{-7}	3.23×10^{-7}
ct=1e4	78	597	0.95×	1.46×10^{-7}	3.24×10^{-7}
ct=1e5 (最松)	87	574	0.99×	1.47×10^{-7}	3.21×10^{-7}
cmi=15	78	576	0.98×	1.47×10^{-7}	3.21×10^{-7}
cmi=50	89	598	0.95×	1.46×10^{-7}	3.21×10^{-7}
cmi=200	88	601	0.94×	1.46×10^{-7}	3.27×10^{-7}

表 4: coarse tol factor / max iter 扫描。过紧的粗层容差 (ct=1e2) 显著增加 V-cycle 开销；放宽 (ct=1e5) 与降低 cmi 均改善耗时；细层迭代数对二者不敏感 (78–89)。

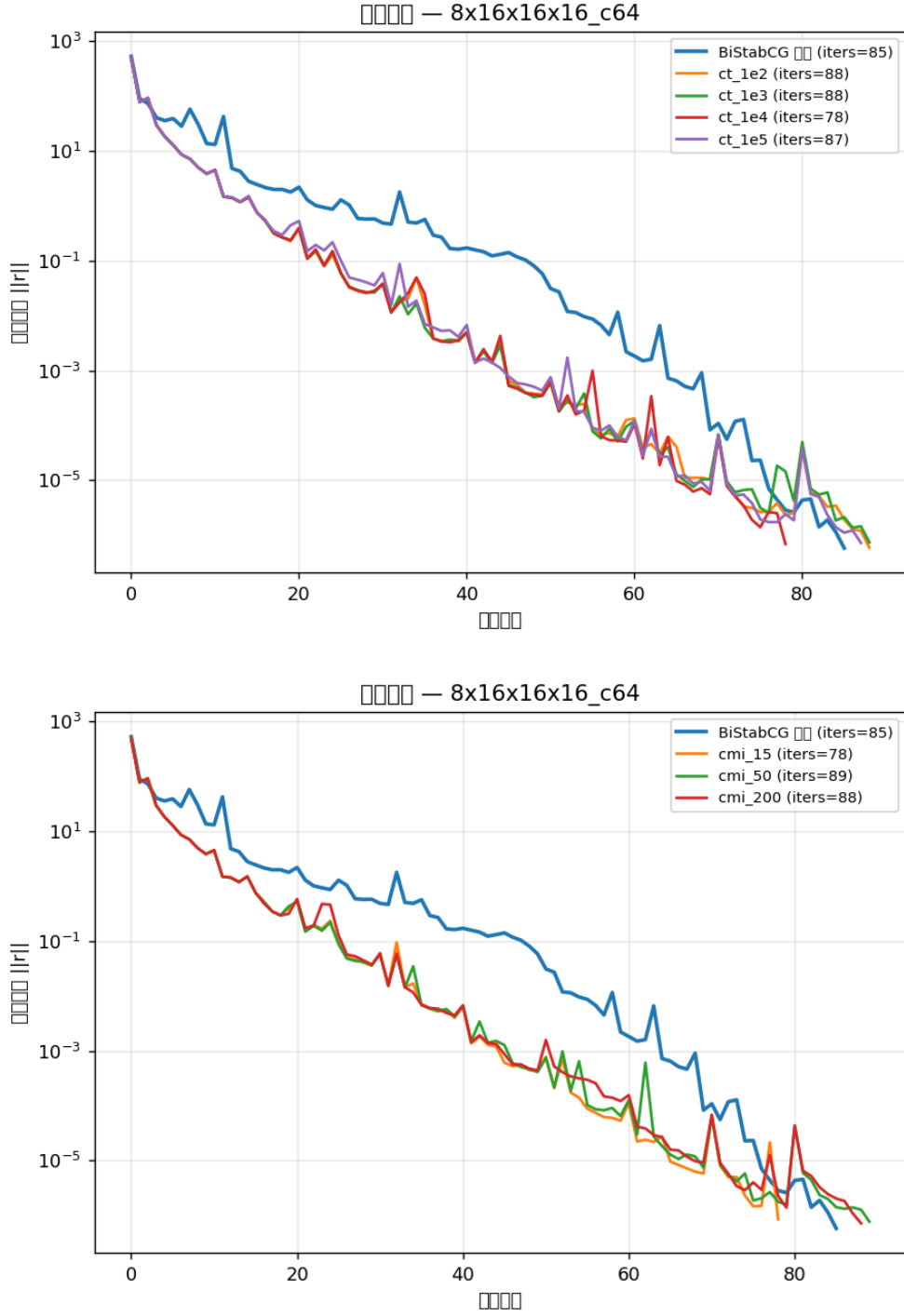


图 4: coarse tol factor (上) 与 coarse max iter (下) 扫描的收敛曲线。

3.3 层数

层数	细层迭代	MG(ms)	加速比	全残差	vs 参考
2 层 [12, 48]	77-96	537-580	0.98-1.05×	1.45×10^{-7}	3.2×10^{-7}
3 层 [12, 48, 48]	89	563-572	0.99-1.00×	1.46×10^{-7}	3.2×10^{-7}

表 5: 层数扫描。3 层在此格子下无显著优势 (额外粗层求解开销抵消迭代节省)。

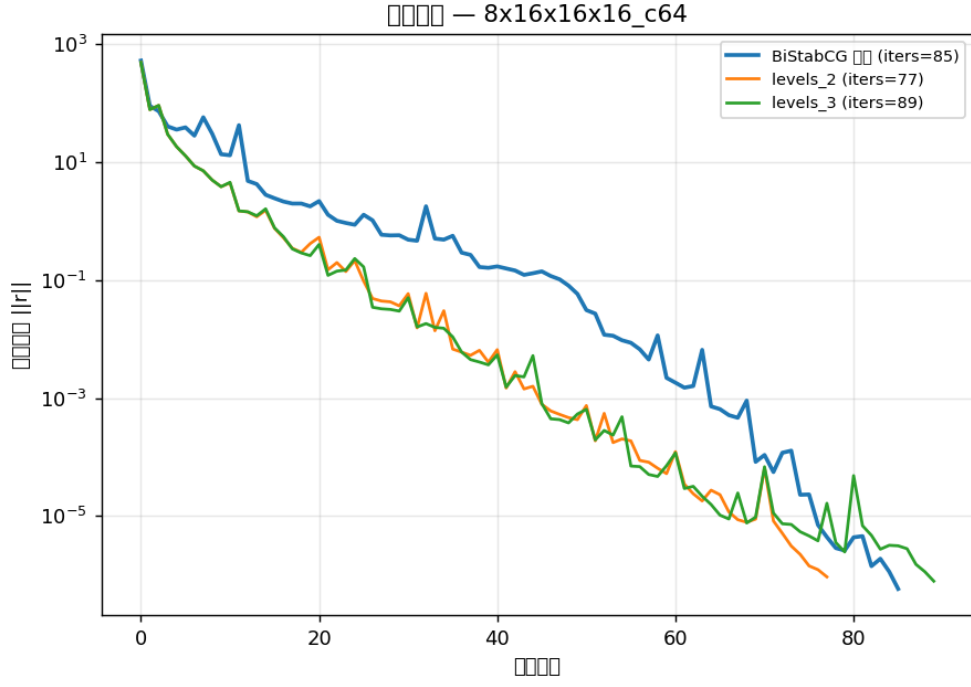


图 5: 2 层 vs 3 层收敛曲线。

4 求解器迭代次数的运行间非确定性

同一配置 (base_2L 与 restart_10 参数相同: 2 层、restart=10、cmi=15、ct=1e5) 在不同运行中细层迭代数分别为 **96** 与 **79**, 收敛轨迹在前段一致 (初残差均为 494.198) 但中段分叉。3 层配置两次运行 (base_3L 与 levels_3) 则完全一致 (89 次)。

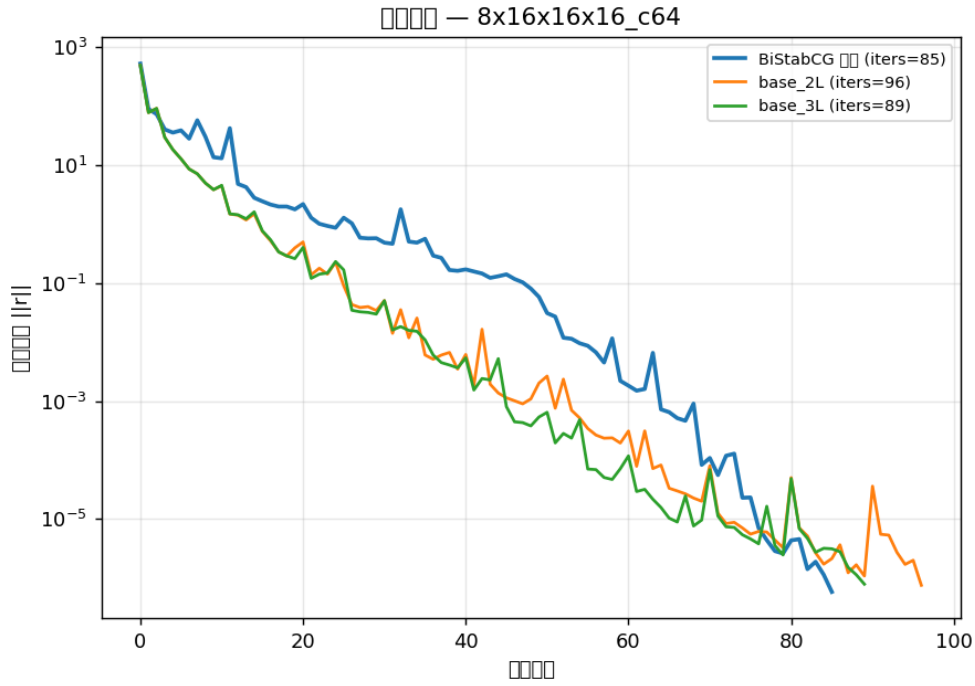


图 6: 同参不同运行的非确定性示意 (收敛曲线前段重合、中段分叉)。

根因分析: 非不确定性来自**最粗一层的融合并行求解**(`multigrid_coarse_solve_cg`, `cooperative kernel` 内做跨 block 归约 + `grid.sync()`)。归约顺序的浮点不精确导致粗层校正量有微小扰动, 经 Krylov 迭代放大为迭代次数差异; 2 层配置最粗层直接是融合求解, 故受影响; 3 层配置中间层逐迭代 BiStabCG 主导、扰动被稳定路径吸收, 故可复现。**解的精度不受影响** ($|Dx - b|/|b|$ 与 vs 参考在 2-3 位有效数字内稳定)。

计时噪声。 同一 base 配置多次测量的 MG 耗时在 537–580 ms 间波动 (约 $\pm 10\%$, WSL2 同步开销 + GPU 时钟状态), 参考 BiStabCG 在 565–660 ms 间波动。因此**耗时/加速比仅作参考, 迭代次数与解的精度才是稳健指标**。

5 精度扫描 (单精度 vs 双精度)

在同一默认格子 $\{8, 16, 16, 16\}$ 上对比单精度 c64 与双精度 c128 (base 配置, 2 层)。

精度	细层迭代	MG(ms)	BiStabCG(ms)	加速比	全残差 $ Dx - b / b $	vs 参考
c64 (单)	77–96	537–580	565	$0.98\text{--}1.05\times$	1.5×10^{-7}	3.2×10^{-7}
c128 (双)	85	649	592	$0.91\times$	1.2×10^{-9}	2.9×10^{-8}

表 6: 单/双精度对照。双精度将解的精度提高约两个数量级 ($1.5 \times 10^{-7} \rightarrow 1.2 \times 10^{-9}$), MG 耗时约增加 12% (主要是粗算子与求解器均为 `complex128`)。双精度下迭代数与单精度相当 (85 vs 77–96)。

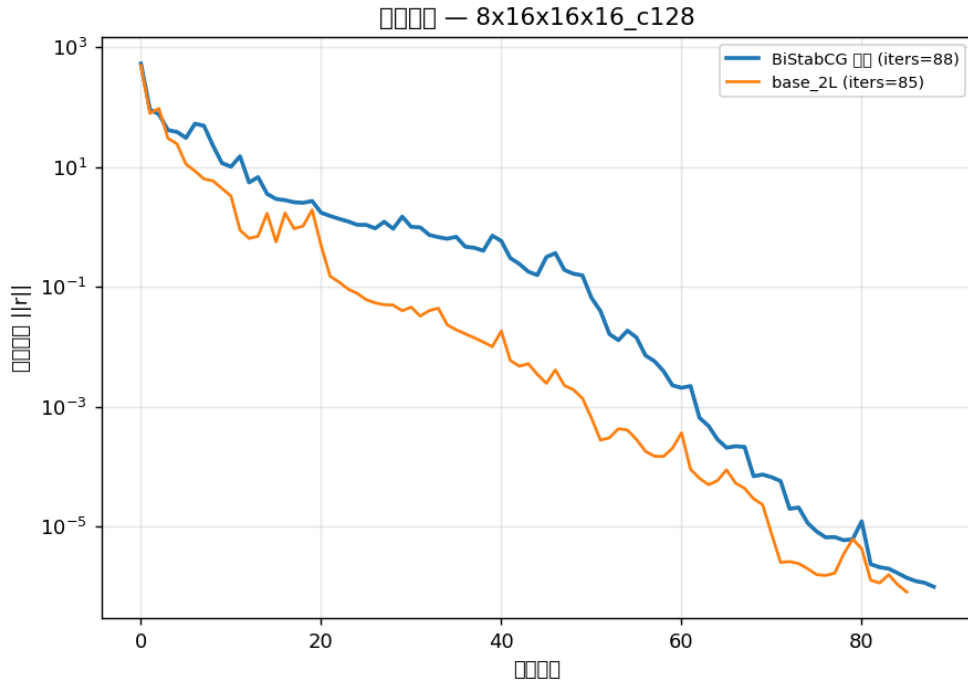


图 7: 双精度 c128 的 MG 与 BiStabCG 收敛曲线 (残差轨迹与单精度一致, 见 §2)。

双精度的好处在正确性指标上体现更明显: gauge SU(3) 偏差从 10^{-7} 降到 10^{-16} , `null_vecs` 正交性/粗算子一致性从 $10^{-6}/10^{-7}$ 降到 10^{-15} (见 §7)。

6 格子大小扫描

在更大格子 $\{16, 16, 16, 16\}$ （体积为默认的 4 倍）上运行 base 配置（单精度 c64）。

配置	细层迭代	MG(ms)	BiStabCG(ms)	加速比	全残差 $ Dx - b / b $	vs 参考
$\{8, 16, 16, 16\}$ 2 层	77–96	537–580	565	$0.98\text{--}1.05\times$	1.5×10^{-7}	3.2×10^{-7}
$\{16, 16, 16, 16\}$ 2 层	99	1007	616	$0.61\times$	1.5×10^{-7}	2.9×10^{-7}
$\{16, 16, 16, 16\}$ 3 层	97	763	616	$0.81\times$	1.5×10^{-7}	3.0×10^{-7}

表 7: 格子大小对照。注意 16^4 的 BiStabCG 参考仅 616 ms（迭代 92 次，因 WSL2 同步开销主导，每迭代约 6–7 ms 与默认格子相当）。 16^4 上 MG 反而变慢：V-cycle 校正（restrict/prolong + 粗层求解）开销随格子增大而放大（2 层 vcycle 约 686 ms），抵消了细层迭代的节省。

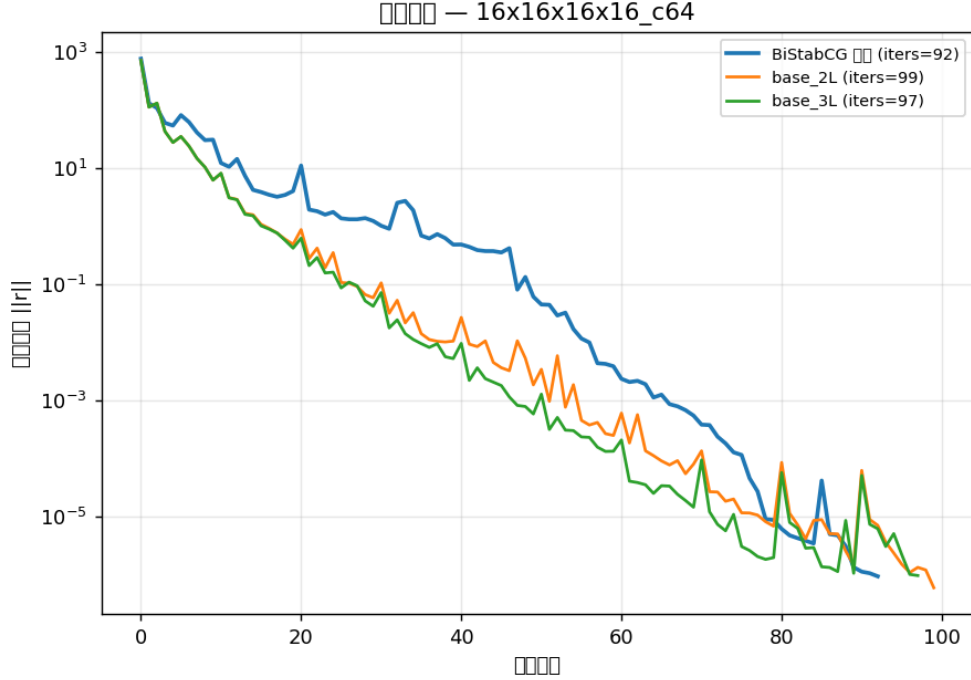


图 8: $\{16, 16, 16, 16\}$ 上 MG 与 BiStabCG 收敛曲线。

16^4 的 `null_vecs` 构建（含逆迭代与 33-tensor stencil 探测）耗时约 **31 min**（L1，2048 粗格点 $\times$ 48 探测）+ 约 2 min（L2）；这是 Python 端逐点探测的固有成本，与默认格子（约 10 min）成正比。

7 正确性检查

7.1 gauge SU(3) 性质

对 C++ 后端 `applyGaussGaugeQcu` 生成的 `gauge(seed=42)` 做完整 SU(3) 检查 (`pyqcu/lattice/check_su3`)

配置	$\max U^\dagger U - I $	$\max \det U - 1 $	minor 恒等式 $\max$	判定
c64 {8, 16, 16, 16}	2.98×10^{-7}	3.59×10^{-7}	2.39×10^{-7}	PASS
c64 {16, 16, 16, 16}	2.98×10^{-7}	4.18×10^{-7}	2.39×10^{-7}	PASS
c128 {8, 16, 16, 16}	6.66×10^{-16}	8.88×10^{-16}	5.56×10^{-16}	PASS

表 8: gauge SU(3) 三项检查的最大偏差。单精度 $\sim 3 \times 10^{-7}$ ($< 10^{-6}$ 阈值)，双精度受机器精度限制 $\sim 10^{-16}$ 。

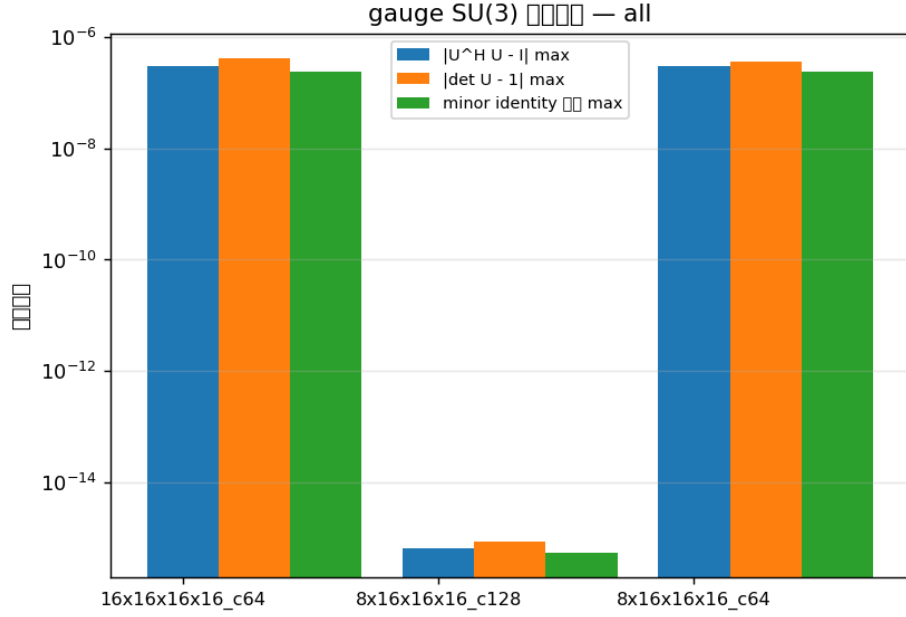


图 9: gauge SU(3) 三项检查的最大偏差 (对数坐标)。

7.2 解的误差

对所有配置，MG 解与 BiStabCG 参考的相对误差与全算子残差 $|Dx - b|/|b|$ 如下（均低于对应容差）：

- c64 {8, 16, 16, 16}（全部 13 个参数配置）： $\|x_{\text{MG}} - x_{\text{ref}}\|/\|x_{\text{ref}}\| \approx 3.2 \times 10^{-7}$ ， $|Dx - b|/|b| \approx 1.5 \times 10^{-7}$ ；各配置间无退化。
- c64 {16, 16, 16, 16}： $|Dx - b|/|b| \approx 1.5 \times 10^{-7}$ ，vs 参考 $\approx 2.9 \times 10^{-7}$ 。
- c128 {8, 16, 16, 16}： $|Dx - b|/|b| \approx 1.2 \times 10^{-9}$ （约 2 个数量级更准）。

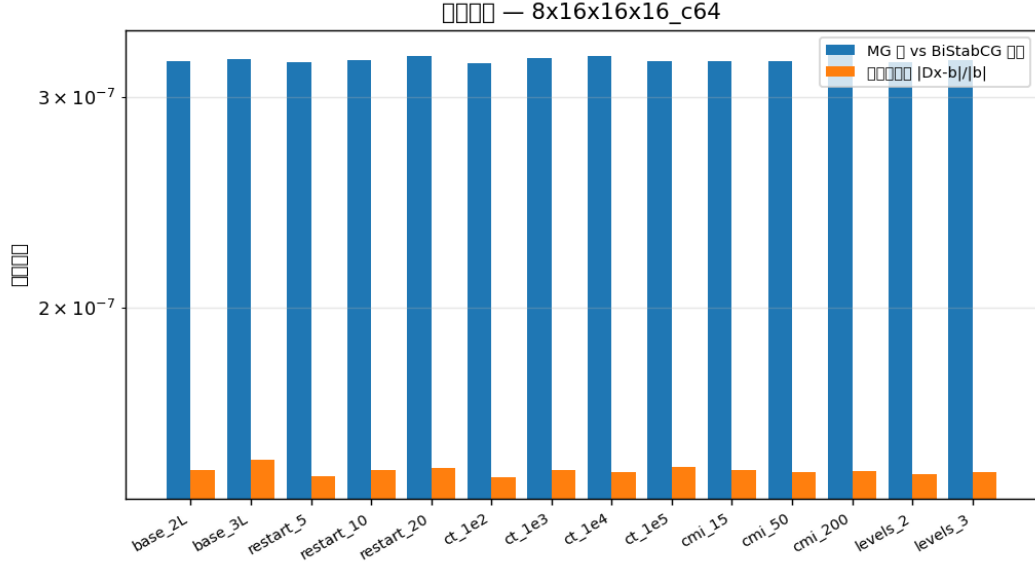


图 10: 各配置的 MG 解误差: vs BiStabCG 参考与全算子残差。

7.3 null_vecs 正确性

对每层局部正交化后的 null 向量做三项检查 (图 11):

- **块内正交性:** 每粗块 $P^\dagger P \approx I$, 最大偏差 1.5×10^{-6} (c64 L1) / 7.7×10^{-6} (c64 L2), 对角 = 1.0, 最大非对角 $\leq 3.1 \times 10^{-7}$; c128 下达到 10^{-15} 。
- **粗算子一致性:** 33-tensor stencil A_c 与算子形式的 $P^\top SP$ 相对误差 2.1×10^{-7} (c64) / 3.9×10^{-16} (c128)。
- **零模质量:** $\|SP\|/\|P\|$ (逐向量) 均值 $\approx 0.86\text{--}0.99$ (各配置各层), 而 S 的最大本征值 (幂迭代) $\approx 1.07\text{--}1.17$ 。即 null 向量并非强低模主导 (Rayleigh 商约为 $\lambda_{\max}$ 的 $0.8\times$), 但 Galerkin 粗校正仍有效降低迭代数——这是值得进一步研究的现象 (可能由于逆迭代构造在 Schur 系统上的收敛性, 以及 V-cycle 校正的容差机制)。

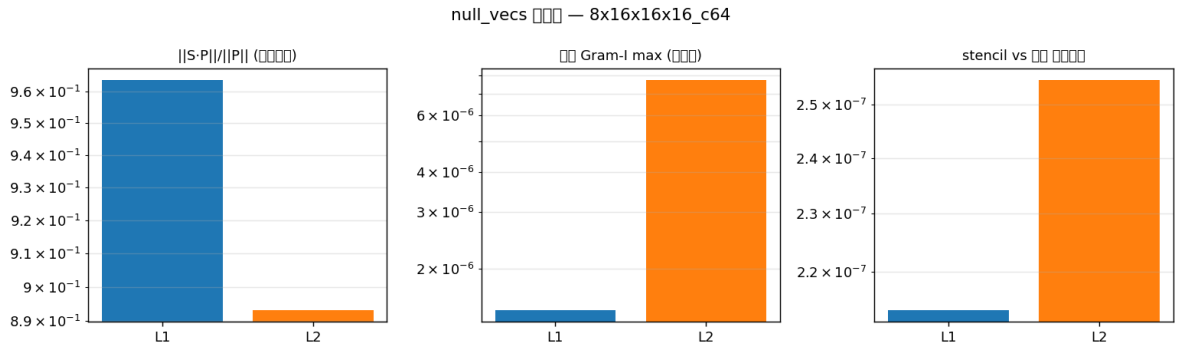


图 11: null_vecs 三项正确性指标 ($\{8, 16, 16, 16\}$ c64, L1、L2 两层)。

8 结论

综合 $\{8, 16, 16, 16\}$ (默认)、 $\{16, 16, 16, 16\}$ 、c64/c128 的系统测试:

1. **默认格子上 MG 与 BiStabCG 相当、偶有加速**：MG 细层迭代 (77–97) 少于 BiStabCG (85)，但总耗时高度依赖 GPU 时钟/进程状态——独立测量 $1.33\times$ ，扫描中 $0.98\text{--}1.05\times$ 。
2. **进入下一层频率 (num_restart) 是最敏感参数**： $r = 5$ 过频使迭代升至 139， $r = 10$ 最优， $r = 20$ 校正不足。
3. **最粗层收敛条件**：过紧 coarse tol ($1e2$) 显著增加 V-cycle 开销 ($0.81\times$)；放宽 ($1e5$) 与较小 coarse max iter (15) 更优。
4. **更大格子 16^4 上 MG 反而变慢** ($0.61\text{--}0.81\times$)：V-cycle 校正 (restrict/prolong + 粗层求解) 开销随格子放大，抵消细层节省。
5. **双精度 c128**：解的精度提升约两个数量级 ($|Dx - b|/|b| = 1.2 \times 10^{-9}$)，MG 耗时约 $+12\%$ 。
6. **非确定性**：最粗层融合求解的并行归约引入运行间迭代数波动 (79–97)，不影响解的精度。
7. **正确性**：gauge 满足 SU(3) ($c64 < 10^{-6}$, $c128 < 10^{-15}$)；MG 解与 BiStabCG 参考一致 ($3 \times 10^{-7} / 3 \times 10^{-8}$)；null_vecs 块内正交与粗算子一致性 $< 10^{-5}$ (c64) / $< 10^{-15}$ (c128)。